

Запускать так: `./run.sh`

Жадники и 2-opt

Сперва напишем жадник, а именно будем проводить ребро в ближайшую ещё не используемую в цикле вершину. Для этого нам нужно посортировать все вершины в порядке отдалённости от каждой конкретной, это мы сделаем явно: выпишем расстояние от вершины u до всех других и отсортируем за $n \log n$, после чего повторим для всех u . Асимптотика такого алгоритма $n^2 \log n$, что может быть слишком дорого для части тестов, потому заведём для себя ограничение на $n^2 \log n < 4 \cdot 10^7$, что должно дать ограничение на 2 секунды работы этой сортировки, решая неравенство $n < 1915.354\dots$, а потому напишем в коде `if` на $n < 1915$, где n — число вершин.

Получив массив расстояний мы написали первую эвристику: выберем некоторую вершину для старта, начнём строить цикл из неё, всё время идя из последней вершины построенного сейчас пути в ближайшую вершину, где цикл ещё не бывал. Такой поиск мы запускаем из всех вершин (формально у этого алгоритма кубическая асимптотика, но на деле она может падать до квадрата и отработала за десятую часть секунду на первых четырёх тестах и за полторы секунды на пятом).

Для шестого теста размер данных слишком велик, чтобы смотреть на все возможные пары, а потому предположим, что не все рёбра могут участвовать в оптимальном гамильтоновом цикле, а именно, отсортируем вершины по x и по y по отдельности и скажем, что вершина u может быть смежна с v только если u лежит в 250 самых ближайших по x или по y после сортировки. Тем самым мы зафиксировали от 250 до 1000 вершин как допустимых соседей. Далее повторим алгоритм, беря ближайшую из вершин, для которых мы посчитали расстояние, если такие вершины закончились, то берём первую, которая ещё не участвует в цикле. Поскольку в этом случае n велико, то мы запускаем жадный поиск цикла не для всех вершин, а лишь для случайно выбранных 500.

Это начальное приближение взяло один тест, а именно третий, получив метрики:

513.61001, 23566.40295, 35394.00872, 45141.33568, 381437.38316, 84125040.77980

что дало 3 балла.

Далее был написан 2-opt, который берёт два ребра и меняет их на диагональные два ребра на тех же четырёх вершинах. Такой алгоритм был запущен на 500 итераций проделать 2-opt, в случае $n > 1915$, который мы считаем более тяжёлым, число итераций было уменьшено до 200, чтобы уложиться во время. Впрочем, алгоритм сошёлся и завершился раньше, чем эти 500 итераций, на всех тестах кроме шестого.

Такой подход уже дал 15 баллов, а именно, собрав 5 порогов на 3, были получены такие метрики:

442.29, 21634.12, 31713.41, 39303.78, 342325.99, 82370058.60

и баллы соответственно равны 3, 3, 3, 3, 3, 0.

Причём решение сильно меняется в зависимости от числа точек, которые я даю своему алгоритму. Попробуем уложившись в минуту найти лучшее сочетание числа итераций 2-opt и окрестности. (причём мы именно отрубали при истечении минуты, так что иногда итерации могли становиться легчевеснее)

После 400 улучшение уже незначительное, зафиксируем его.

окрестность в 1 сторону	число итераций	метрика
100	700	101471431.11
150	277	88531528.41
200	391	83768532.93
250	305	83347373.48
300	342	81301047.42
350	308	81062475.19
400	398	79827426.05
450	254	79531826.14

Лин Кёрниган

Перед семинаром много кто говорил, что их tsp хорошо взяло решение из статьи с генетикой и Лином Кёрнигом, будем двигаться в его сторону, пока напишем Лина Кёрнига.

Его мы реализовали следующим образом: выберем ребро $\{v, w\}$ и представим маршрут как $(w, u_1, u_2, \dots, u_n, v)$. Попытаемся найти другое ребро из v , пусть это ребро $\{v, w'\}$, w'' — следующая после w' на пути. Тогда заменим цикл который был на цикл $(w, w'', \dots, u_n, v, w', \dots, u_1, w)$. Что пока что 2-opt, но мы возьмём такую замену с одной из первых *search* идущих впереди по циклу вершин, из которых возьмём минимизирующую, даже если изменение отрицательной полезности. Значение *search* для всех тестов кроме шестого вполне подойдёт и равно числу вершин, то есть что можно пробежаться по всему графу, на последнем же тесте это дорого. Также можно настроить число итераций которые мы пытаемся проделать, пока ищем улучшение, чтобы не ходить слишком долго. Пока что зафиксируем ограничение делать Кёрнига Лина не дольше 30 секунд.

Пока будем настраивать 5 тестов, потому что шестой работает слишком долго

search	тест 1	тест 2	тест 3	тест 4	тест 5
5	433.07	21391.17	31093.05	38785.47	337643.44
20	433.07	21391.17	31093.05	38785.47	337643.44
200	433.07	21391.17	31093.05	38785.47	337643.44
2000	433.07	21391.17	31093.05	38785.47	337643.44

То есть метрики не меняются и лишняя свобода тут не к чему. Выбрав $search = 10$ как запас для большого теста запустим алгоритм на всех шести тестах, пока с *neib* — числом видимых вершин впереди равным 2000. Это дало метрики

433.07, 21391.17, 31093.05, 38785.47, 337643.44, 74302475.01

Что даёт 18 баллов, взяв все первые пороги. Чуть лучше получаются метрики при возможности обходить в Кёрниге Лине путь в 2 стороны:

434.45, 21391.17, 30833.25, 38568.74, 333689.45, 74443726.99

Также все первые пороги.

Генетика с меметикой

Напишем алгоритм из статьи, которую присылали в чат.

В реализации появились следующие параметры: размер популяции, количество скрещиваний, вероятность мутации и после какого уровня совпадения мы считаем популяцию выродившейся.

Генетика получилась тяжеловесная для пятого и шестого теста, так что сперва отдельно настроим её для первых четырёх тестов, где выполняется много итераций.

Проставим стартовые 20 путей популяции, 10 скрещиваний, 20% для мутации и считать, что мы сошлись при хэмминговом расстоянии между всеми не больше 0.05 от длины. На них метрики составили:

428.87, 20750.76, 29450.50, 37272.91

Что дало все пороги на 5 на этих тестах, но нам важно сходиться быстрее. Сперва поймём что значит большая вероятность мутации и как она влияет на происходящее (на четвёртом тесте будем брать значение на 200 эпохе для ускорения процесса).

вероятность мутации	тест 1	тест 2	тест 3	тест 4
0	428.98	20750.76	29569.74	37527.79
0.3	428.87	20750.76	29440.41	37347.72
0.6	428.87	20750.76	29450.50	37446.26
0.9	428.87	20750.76	29450.50	37437.95

При увеличении вероятности мутации затрачиваемое время, ожидаемо, увеличилось, причём время до получения хорошего ответа скорее уменьшилось, а вот со сходимостью возникли проблемы: делалось очень много итераций без улучшения ответа перед тем, как популяция всё таки вырождалась. Иногда приходилось даже останавливать генетику вручную. Для того чтобы быстрее пройти стартовые значения и добавить больше вариативности стоит взять вероятность побольше, а чтобы сойтись поменьше. Так и поступим начнём в 0.55 (далее улучшения не было), и каждые 100 эпох будем уменьшать на 0.15, пока не дойдём до 0.1, ниже которого опускать не будем.

Так метрики составляют

428.98, 20750.76, 29450.50, 37236.04

Следующей немаловажной проблемой является то, что сейчас во всех случаях алгоритм останавливается только при полном совпадении популяции, то есть не нашлось ещё популяций, которые бы ещё не совпадали, то их хэммингово расстояние было в пределах 5% друг от друга. Посмотрим что самое большое можно выбрать так, чтобы не ухудшить ответ.¹

хэммингов диаметр	тест 1	тест 2	тест 3	тест 4
0.05	428.98	20750.76	29450.50	37236.04
0.2	428.87	20750.76	29450.91	37236.04
0.45	428.87	20750.76	29450.50	37236.04
0.6	428.98	20750.76	29469.36	37236.04
1	428.98	20750.76	29637.47	37335.04

В первом и втором случае завершения снова происходили только при нулевом расстоянии, в третьем среднее при завершении уже было между 2 и 6, что в целом уже не даёт пользы от скрещивания: меняются 2-6 рёбер. На 1 и 0.6 метрики повредились уже сильно, а вот в остальных случаях всё было в порядке. Возьмём 0.45 (т.е. радиус от случайного элемента в 0.225).

¹Примечание: в решении считается расстояние от случайного вектора до всех, тем самым диаметр ограничивается как радиус от случайного, поэтому диаметр = 1, это не тоже самое, что кто угодно вырожден.

Теперь выберем оптимальное значение отношения числа мутаций к размеру популяции. При слишком большом значении от популяции ничего не будет оставаться и можно потерять хорошие решения, заменив их скрещенными плохими, а при маленьком отношении генетике сложно остановиться, так как очень много наилучших решений долго остаются в популяции.

число скрещиваний (при 20 путях)	тест 1	тест 2	тест 3	тест 4
1	428.98	20852.28	29546.16	37631.31
5	428.87	20750.76	29494.42	37241.52
10	428.87	20750.76	29450.50	37236.04
15	428.87	20750.76	29450.50	TL
20	428.87	20750.76	29512.56	TL ²

При 20 возникла проблема, что популяция не вырождалась, а именно мгновенная подмена всей популяцией, часть из которых мутировала не давало хеминговой метрике упасть: видимо мутации портят хемингову метрику, генетику останавливали вручную. Аналогичная проблема была при 15 на первом и третьем тесте. При маленьких значениях итераций стало больше, они стали быстрее, и мы стали вырождаться на худшей метрике. Оставим 1/2.

И наконец настроим количество элементов (то есть путей) в генетике. Слишком маленькое даст мало вариативности вначале и популяции будет проще выродиться, а слишком большое даст слишком тяжёлый шаг и мы не вложимся во время.

число путей	тест 1	тест 2	тест 3	тест 4
4	428.87	20852.28	29650.65	37578.34
10	428.87	20750.76	29493.36	37459.97
20	428.87	20750.76	29450.50	37236.04
40	428.87	20750.76	29450.50	TL
100	428.87	20750.76	29440.82 (TL)	TL

До 20 метрики получились хуже, чем было. После же не дали улучшений, замедлив время. Оставим как есть.

Запустим то что есть на пятом и шестом тесте и посмотрим сумеет ли оно сделать хотя бы несколько итераций генетики. Для 20 и 10 не сумело, а для 4 путей в популяции на пятом тесте алгоритм получил 324279.89, то есть 3 балла, на шестом 72187749.03, сделав 2 итерации за 6 минут, и не улучшив метрики.

То есть на данный момент решение берёт 26 баллов так: 5, 5, 5, 5, 3, 3.

Отжиг

Попробуем написать легковесный отжиг, чтобы не запускать медленную генетику. У нас была хорошая вероятностная мутация, которая, как мы видели при настройке генетики, даёт очень многое, ведь метрика не сильно менялась от числа скрещиваний, но сильно от вероятности мутации.

Сделаем отжиг с двухступенчатой мутацией: сделать случайную замену 4 рёбер + применить Лина Кернигана.

Настроим коэффициенты как и в предыдущих отжигах, а именно подберём стартовую и конечную температуру так, чтобы на начальных итерациях вероятность принять ошибку была немного больше 90%, а на последних меньше 1%, после чего выберем переход, чтобы итераций было как можно больше, но мы уложились во время. Поскольку пятый и шестой тест ощутимо дороже остальных сделаем другие параметры при числе вершин больше 1000.

Запустив пробный отжиг с очень большими границами видно, что дельта бывает в границах от 0.1 до 100, в зависимости от теста и структуры улучшения, на больших тестах это значение увеличится. Откуда нет смысла в слишком маленьких температурах и 10^{-4} будет вполне достаточно.

В этом случае метрики, даже при отключённой генетике, составили

428.87, 20750.76, 29440.41, 37188.45, TL , TL .

То есть метрики даже улучшились по сравнению с генетикой. Единственная проблема — скорость, сейчас ещё можно оптимизировать скорость Лина Кернигана, так как сейчас мы пытаемся применить улучшение для всех вершин, а можно для случайных.

Определим сколько итераций достаточно для того, чтобы не терять точность.

число итераций	тест 1	тест 2	тест 3	тест 4	тест 5
10	428.87	20750.76	30086.57	38568.74	333689.45
20	428.87	20750.76	29613.85	38567.56	333685.23
50	428.87	20750.76	29440.41	37726.38	330012.37
100	428.87	20750.76	29450.50	37576.81	TL
250	428.87	20750.76	29465.50	37214.78	TL

При 100 итерациях решение уже не уступает в баллах, хоть и проигрывает немного по метрикам. Однако для больших тестов (пятого и шестого) такая оптимизация всё равно не подходит и нужно что-то более легковесное.

POPMUSIC и частичная перестройка

Откажется от тяжеловесных генетики и отжига и попытаемся улучшить жадник, методами POPMUSIC. Будет действовать подобно тому, что изложено в книге “Design of Heuristic Algorithms for Hard Optimization”.

Выделим несколько подряд идущих вершин в текущем пути, попытаемся поменять их порядок так, чтобы улучшить расстояние, если нашлась улучшающая перестановка, то представим вершины, после чего независимо от того были изменения или нет сдвинемся на пол длины блока вперёд.

Рассмотрим несколько вариантов реализации, в первом возьмём блок маленькой длины, при которой можно рассмотреть все возможные перестановки и выбрать оптимальную. Максимальный размер блока, при котором мы не получаем TL это 9, на нём метрики составили

439.67, 22688.71, 32352.80, 43065.67, 371380.35, 97226944.13

Что даёт 12 баллов как 3, 3, 3, 0, 3, 0.

Будем смотреть на окна побольше, и подобно тому, что было предложено в статье попробуем перерешивать блок побольше с помощью k -opt.

Написав 2-opt метрики составили

439.93, 21431.23, 31797.34, 38908.87, 336835.25, 79015173.08

Что дало 15 баллов 3, 3, 3, 3, 3, 0. Причём время на первый проход 2-opt был значительно больше, чем на последующие. Для того, чтобы можно было ставить окно побольше (было 3000), попытаемся сперва предобработать данные, а именно сперва сделать более маленькие 2-опты, чтобы не рассматривать их позже. Поставим интервал в 5000 и подберём первые несколько проходов так, чтобы время работы было приемлемым.

Подобрав около случайным образом два запуска на 200 и на 1000 удалось взять первый порог на шестом тесте.

439.93, 21431.23, 31797.34, 38908.87, 336835.25, 77627972.78

18 баллов, все первые пороги.

Также было добавлено и 3-opt, которое работало значительно дольше, метрики для 2-opt и 3-opt вместе составили:

438.67, 21431.23, 31720.52, 38882.71, 336713.56, 77754826.33

В нынешнем решении есть следующая проблема: около 1% всех рёбер дают около 10% от пути, то есть есть часть рёбер, выбранных неудачно в начале, которые нельзя убрать в терминах небольшого блока. Для того, чтобы с этим бороться добавим перестройку, которая попытается искоренить эту проблему.

Найдём альтернативу получше, для маленькой части рёбер, для этого найдём ту долю рёбер, которые нам кажутся слишком большими и разрушим, после чего попытаемся найти наилучшую склейку имеющихся кусков. Это будет задача ассиметрического tsp на небольшом количестве вершин, которую можно решить каким-то из ранее предложенных методов, например отжигом, как самым быстрым и лучшим из них.

Для асимметрического tsp делать Кёрнигана-Лина затратнее, так как разворачивается часть цикла, что может поменять метрики, такая реализация отжига даёт TL на шестом тесте.

Попробуем вместо дожатия пути Кёрнигоном Лином дожимать максимальным 2-opt. Такая реализация дала метрики

428.87, 20750.76, 30873.60, 38882.71, 336713.56, 77319205.04(TL)

Что берёт 5, 5, 3, 3, 3, 3. Однако алгоритм затевался для больших тестов, так что нас не интересует.

Можно заменить отжиг на простой максимальный 2-opt, метрики у такого подхода будут следующие:

438.67, 21431.23, 31720.52, 38882.71, 336713.56, 76354942.06

Все нижние пороги. То есть для больших тестов результаты хуже, чем у Лина-Кёрнигана, который всегда берёт лучшее ребро из имеющихся.